

Gitlab CI/CD Pipeline

Casey Stijlaart

Table of Contents

- 1. Overview 1
- 2. Custom Docker Image 1
- 3. Pipeline Configuration File 1
 - 3.1. Key Features..... 1
 - 3.2. Example Output 1
- 4. Conclusion 2

1. Overview

The GitLab CI/CD pipeline for your embedded software project is designed to ensure code quality, generate documentation, and perform rigorous testing. The pipeline consists of three stages: build, testing, and docs. In the docs stage, documentation is generated, and in the testing stage, various tests, including Google tests and code style checks, are executed.

2. Custom Docker Image

To accommodate MongoDB drivers for testing, a custom Docker image (caseystijlaart/iotdatalab-image) is created. This image includes the necessary dependencies and configurations for MongoDB. The image is pulled and used in the testing job.

3. Pipeline Configuration File

3.1. Key Features

3.1.1. Documentation Generation

1. Documentation is generated for the project plan, system design, and technical document.
2. Documentation jobs are triggered manually or when there are changes in relevant documentation files.

3.1.2. Google Test Cases

1. Google test cases are run on the embedded software.
2. Code coverage is checked, and the pipeline fails if coverage falls below the specified threshold of 85%.

3.1.3. Code Style Check:

1. Code style is checked using cppcheck.

3.2. Example Output

To showcase how the pipeline works regarding to guarding the software based on coverage, I have disabled multiple tests, to let the coverage get too low, see [\[Example Output Pipeline Coverage Failure\]](#).

```

$ lcov --list hal_coverage.info
Reading tracefile hal_coverage.info

Filename                                |Lines   |Functions|Branches
                                      |Rate    Num|Rate    Num|Rate    Num
=====
[/builds/t6-rb/fall-23/iot-measurement-setup/iot-measurement-setup-for-observing-lab-set-ups/src/system/libs/hal/]
src/Database.cpp                        |52.2%    69|50.0%    6|-      0
src/DatabaseDataFormatter.cpp           |100%     16|100%     2|-      0
tests/DatabaseDataFormatterTest.cpp      |100%     48|100%    37|-      0
tests/DatabaseTest.cpp                  |100%     19|100%    21|-      0
=====
Total: |78.3%   152|95.5%   66|-      0

$ COVERAGE_THRESHOLD=85 # collapsed multi-line command
Code coverage: 78.3%
Error: Code coverage is below the threshold of 85%
Cleaning up project directory and file based variables
ERROR: Job failed: exit code 1

```

Figure 1. Example Output Pipeline Coverage Failure

4. Conclusion

This GitLab CI/CD pipeline ensures comprehensive testing, documentation generation, and code style checks for your embedded software project. The modular structure allows for flexibility and easy maintenance, and the pipeline is configured to meet the specific needs of your project. Regularly reviewing and updating the pipeline ensures continuous improvement in code quality and development efficiency.